```
number = 4
double = (numbers) -> number * 2 for number in numbers
double [3,4,5,6]

number
# 6
```

The value of this outer variable has been changed by the comprehension.

Comprehensions in CoffeeScript have side effects. The names used are variables and so they use the same scoping rules and implicit declaration as any other variables in CoffeeScript. The ECMAScript 6 `let` expression is designed for names that should *only* apply to a block of code, just like the comprehension shown previously:

```
if (something) {
  let x = 3;
}
```

ECMAScript 6

If you don't like comprehensions to have side effects, you can see how that could be useful. How can you use it in CoffeeScript? Well, the brevity and flexibility of CoffeeScript's syntax mean that you can approximate `let` with a simple `do _>` form:

```
number = 5
do (number=0) -> number for number in numbers
```

Function parameters always *shadow*, so there's no way to assign a value to the `number` variable. This means you can rewrite the earlier example so that the comprehension doesn't clobber the outer variable:

```
number = 4
double = do (number=0) ->
  (numbers) -> number * 2 for number in numbers
double [3,4,5,6]
number
# 4
```

CoffeeScript favors simplicity. Sometimes when you think you need a specific new feature, there's an easy way to achieve the result you want without it. It's possible to write entire CoffeeScript programs that use only parameters and never use variables.